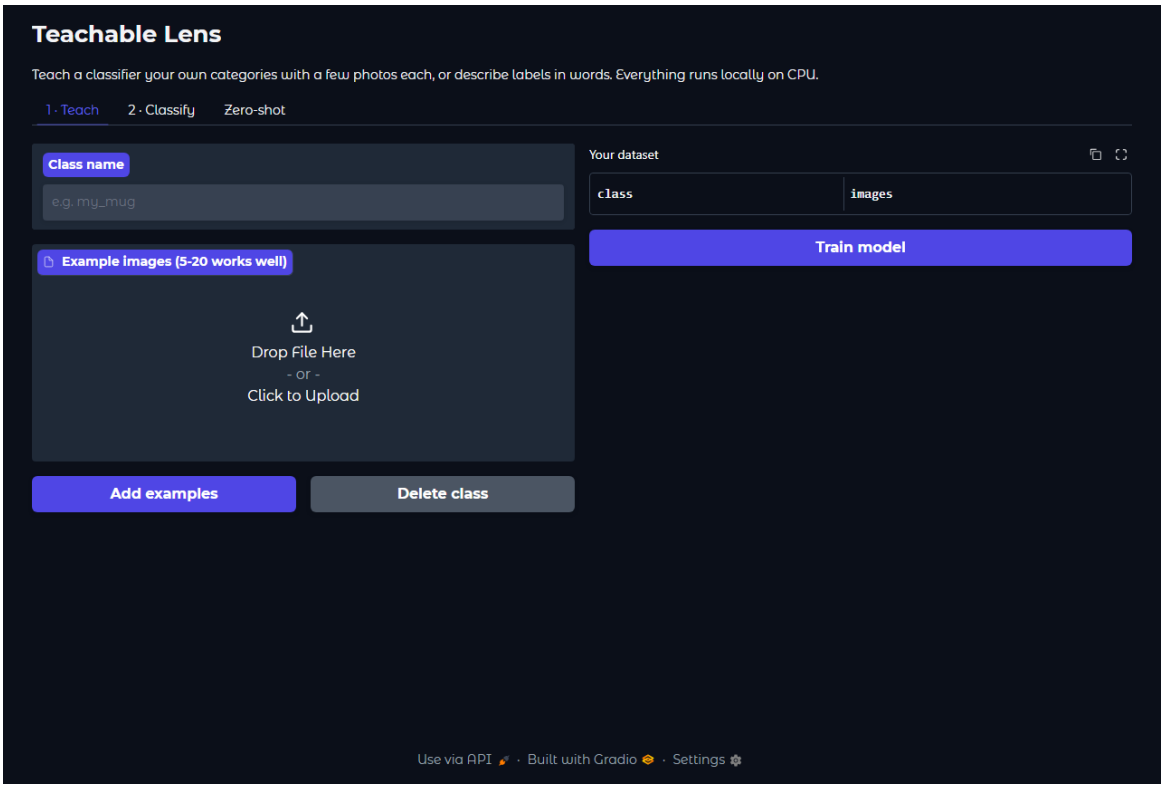
Teachable Lens

A local, few-shot image classifier built on CLIP (ONNX) with a Gradio UI



The Teach tab of the running application

Project report - September 2026

1. Introduction

Most image-classification projects start from a fixed dataset such as CIFAR-10 or MNIST and train a network that recognises exactly those classes. Teachable Lens takes the opposite direction: the user decides what the classes are at run time. You create a class, drop in a handful of example photos (your mug, your cat, a hand-drawn logo), press **Train**, and a new classifier is ready in about a second. When you have no examples at all, a **zero-shot** mode classifies an image against labels typed as plain text.

The idea behind this is transfer learning in its simplest form. A large pretrained vision-language model, OpenAI's CLIP, is used only as a frozen feature extractor. It maps any image to a 512-number vector in which visually and semantically similar images lie close together. A small logistic-regression layer trained on those vectors is enough to separate the user's classes.

1.1 Goals

- Let non-experts build their own classifier with 5-20 images per class.
- Run fully offline after one model download, on CPU only.
- Stay within a laptop budget of 4 GB VRAM and 16 GB RAM with no PyTorch or TensorFlow install.
- Provide a clean browser UI with upload, webcam and clipboard input.

1.2 What makes it different

Compared with a typical "train a CNN on a dataset" project, Teachable Lens has no fixed label set, needs no GPU, trains in milliseconds rather than hours and combines two classification modes (example-based and language-based) over a single shared encoder.

2. Hardware constraints and design decisions

The target machine has a 4 GB GPU and 16 GB of system memory. A standard PyTorch install with CUDA takes several gigabytes on disk, and fine-tuning even a small vision transformer in 4 GB of VRAM is tight. These constraints shaped every choice:

Decision	Choice	Reason
Inference engine	onnxruntime (CPU)	Small wheel, no CUDA or PyTorch needed
Backbone	CLIP ViT-B/32, int8 ONNX	Strong general features, small quantized file
Training	Frozen encoder + LogReg	Trains on CPU in milliseconds, no GPU memory
Tokenizer	HF tokenizers (Rust)	Avoids installing the full transformers library
UI	Gradio 6	Browser UI with webcam/clipboard in pure Python
Storage	Plain folders + joblib	Dataset stays human-readable and portable

3. System architecture

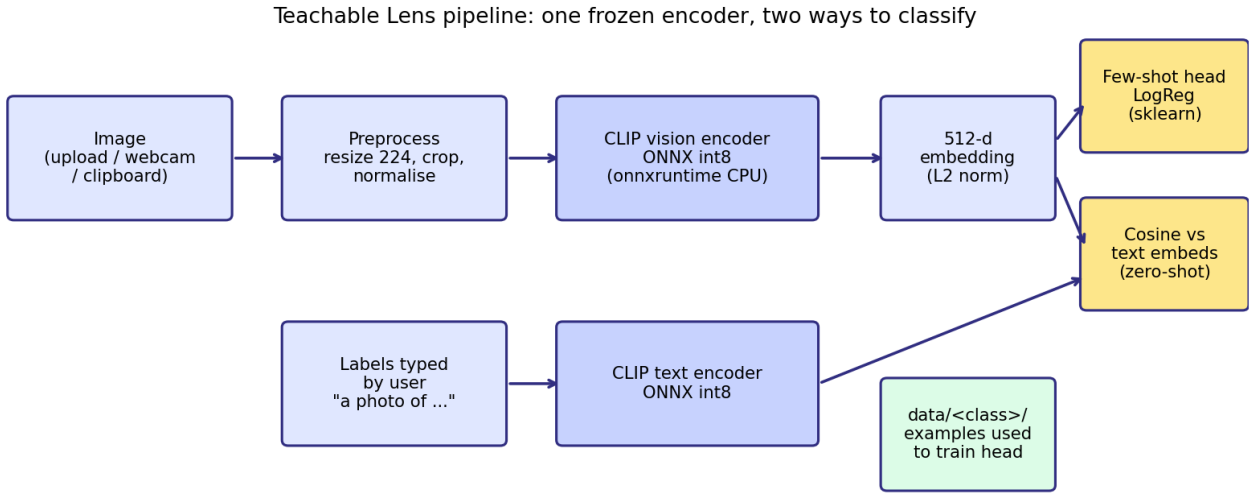


Figure 1. Data flow. Both modes share the same frozen CLIP encoder.

An input image is resized so its shorter side is 224 pixels, center-cropped to 224x224 and normalised with CLIP's channel mean and standard deviation. The quantized vision encoder produces a 512-dimensional embedding that is L2-normalised. From there the pipeline splits:

Few-shot mode. Embeddings of all stored example images train a multinomial logistic-regression classifier (scikit-learn, C = 10, class-balanced weights). The trained head is saved to `models/head.joblib` and reloaded on start-up.

Zero-shot mode. Each user label is wrapped in the prompt template "a photo of {label}" and encoded by the CLIP text encoder. Cosine similarities between image and text embeddings, scaled by CLIP's logit factor of 100, pass through a softmax to give probabilities.

4. Models used (Hugging Face)

All pretrained weights come from the Hugging Face Hub and are downloaded once into the local cache (`~/.cache/huggingface`). No model is trained from scratch; only the small classification head is trained locally.

Component	Hugging Face repo / file	Size
Vision encoder	Xenova/clip-vit-base-patch32 onnx/vision_model_quantized.onnx	85 MB
Text encoder	Xenova/clip-vit-base-patch32 onnx/text_model_quantized.onnx	62 MB
Tokenizer	Xenova/clip-vit-base-patch32 tokenizer.json	< 5 MB
Original weights	openai/clip-vit-base-patch32 (MIT licence)	-

The Xenova repository is a community ONNX export of the official OpenAI checkpoint with 8-bit weight quantization. Quantization cuts the file size roughly by four and speeds up CPU inference at the cost of a small drop in accuracy.

5. Implementation

The code is split into three short modules, about 250 lines in total.

File	Responsibility
lens/clip_onnx.py	Downloads the ONNX files, preprocessing, image and text embedding
lens/head.py	FewShotHead: fit, predict, save, load (logistic regression)
app.py	Gradio UI, dataset folder management, training and classify callbacks

5.1 Embedding an image without PyTorch

CLIP's preprocessing is reimplemented with Pillow and NumPy, so the only runtime dependency for the model is onnxruntime:

```
img = resize(img, shorter_side=224, BICUBIC)
img = center_crop(img, 224)
x = (pixels / 255 - MEAN) / STD  # CLIP constants
emb = session.run(['image_embeds'], {'pixel_values': x})
emb = emb / norm(emb)
```

5.2 Training the head

Training embeds every image under `data/<class>/` in batches of 16 and fits the logistic regression. Because the encoder is frozen, the heavy computation happens exactly once per image; fitting the head itself took **8 ms** for 30 images in our measurement. Adding a new class means adding a folder and retraining, with no risk of forgetting old classes.

5.3 Dataset storage

Examples are copied into one folder per class with sequential file names. The dataset can be inspected, backed up or edited in a normal file manager, and the Teach tab shows a live count of images per class.

6. User interface

The Gradio interface has three tabs. **Teach** manages classes and triggers training. **Classify** runs the trained model on an uploaded, webcam or pasted image and shows the top-5 classes as confidence bars that update as soon as the image changes. **Zero-shot** accepts a comma-separated list of labels and needs no training at all. Errors such as training with only one class are shown as toast messages instead of crashing the app. The title page shows the Teach tab.

7. Evaluation

To get reproducible numbers without shipping a third-party dataset, a synthetic benchmark was generated: three shape classes (circle, square, triangle) drawn with a random colour out of ten, random size, random position and one of four background tints. Colour is deliberately uninformative, so the classifier has to learn shape. The test set holds 90 images (30 per class) that never appear in training.

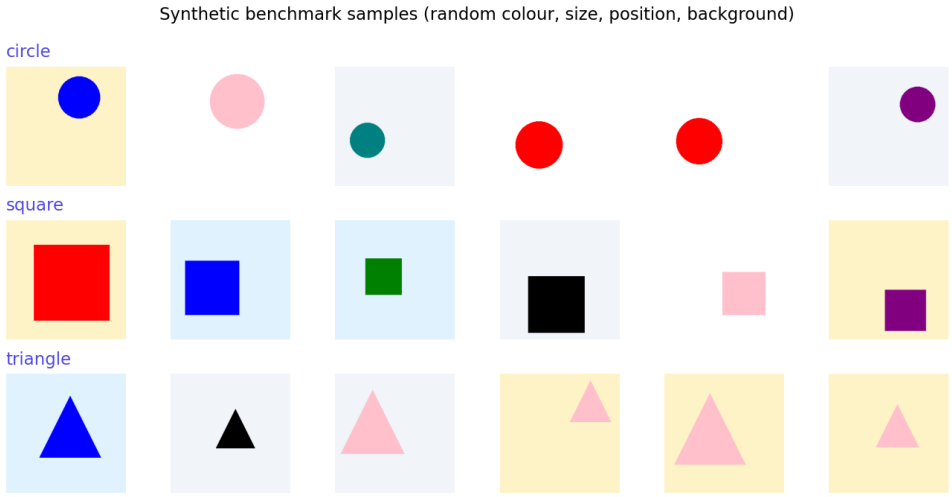


Figure 2. Example benchmark images.

7.1 Accuracy versus number of examples

For each budget of k images per class, five random training subsets were drawn from a pool of 20 per class. Figure 3 reports mean and standard deviation of test accuracy.

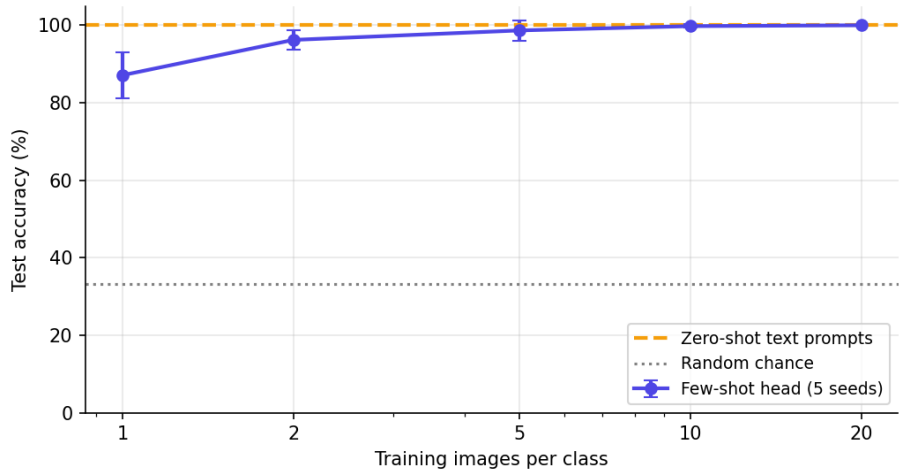


Figure 3. Few-shot accuracy vs. examples per class, with zero-shot baseline.

Images / class	1	2	5	10	20
Accuracy (mean)	87.1%	96.2%	98.7%	99.8%	100.0%
Std. dev.	+/-5.9%	+/-2.5%	+/-2.7%	+/-0.4%	+/-0.0%

7.2 Zero-shot versus few-shot

With the prompts "a photo of a circle / square / triangle" and no training, the zero-shot mode reached **100.0%** accuracy. The few-shot head with 10 examples per class reached **100.0%**. Both modes classify this benchmark almost perfectly. Simple shapes have short, precise names that CLIP learned from its training captions, so the text prompts alone are enough.

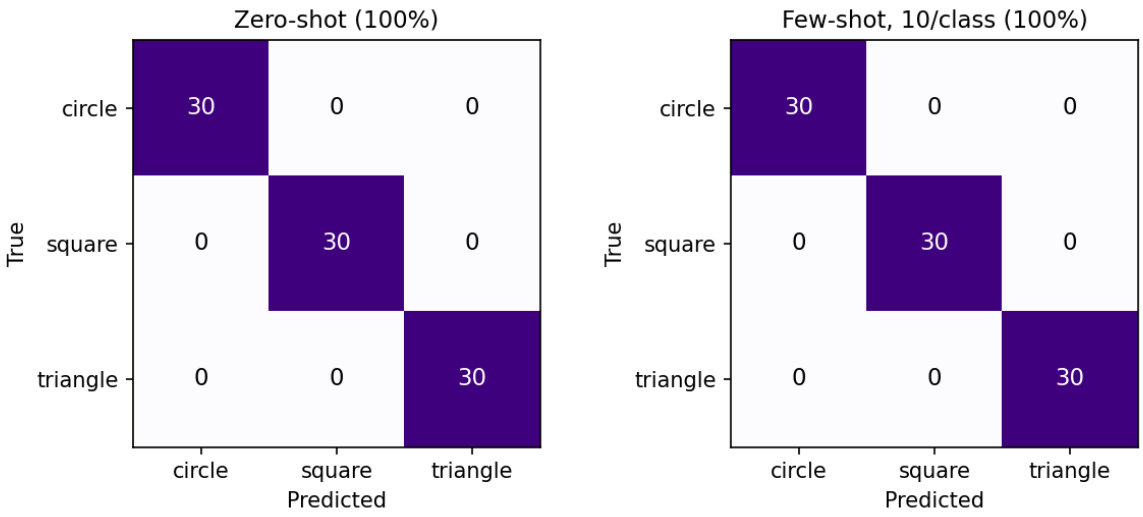


Figure 4. Confusion matrices on the 90-image test set.

Zero-shot works when a category has a clear everyday name and needs nothing but words. Few-shot is what makes the app personal: categories like "my bike" versus "my friend's bike", or a specific logo, cannot be described in text but can be learned from a few photos. The two modes complement each other in the same app.

7.3 Qualitative test

During development, a red circle was classified zero-shot against "a red circle", "a blue square" and "a green tree" with 99.7% confidence for the correct label. A head trained on four circles and four squares in red, blue, green and orange classified an unseen *purple* square as square with 67% probability, showing that it generalises to colours it never saw.

7.4 Limits of this benchmark

Synthetic shapes are much simpler than real photos, so these numbers show that the pipeline works and how accuracy scales with examples; they do not predict accuracy on arbitrary user data. Evaluating on a real few-shot benchmark would be a natural next step.

8. Performance on the target hardware

All timings were measured on the development laptop using onnxruntime's CPU execution provider, after one warm-up call.

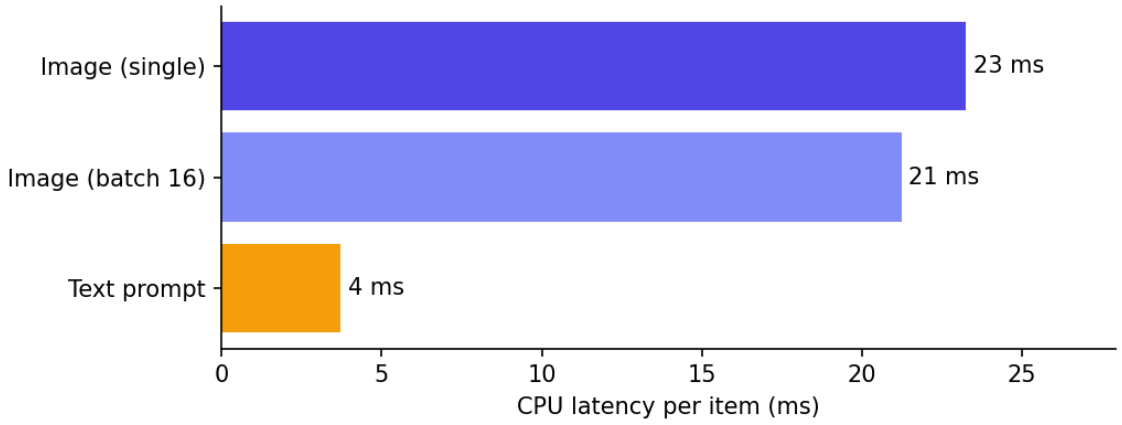


Figure 5. Average CPU latency per item.

Metric	Value
Image embedding, single	23 ms
Image embedding, batch of 16	21 ms / image
Text embedding (one prompt)	4 ms
Head training, 30 images	8 ms (after embedding)
Process memory (RSS) after benchmark	475 MB
GPU memory used	0 MB (CPU only)
Model download (one time)	147 MB

The whole application fits comfortably in the 16 GB RAM budget and leaves the 4 GB GPU free. Latency is low enough for the Classify tab to update interactively when an image is uploaded.

9. Limitations and future work

- **Fine-grained classes.** Frozen CLIP features can struggle with very similar classes (two nearly identical products); fine-tuning the encoder would help but needs a GPU framework.
- **Quantization.** int8 weights trade some accuracy for speed; an fp32 ONNX file can be swapped in by changing one constant.
- **GPU acceleration.** Installing onnxruntime-directml would let Windows use the 4 GB GPU without PyTorch.
- **Hybrid mode.** Text embeddings of class names could be added as extra training points so classes work well with only one or two photos.
- **Export.** The trained head could be exported to ONNX (skl2onnx) for use on mobile or the web.

10. How to run

```
pip install -r requirements.txt
python app.py
# open http://127.0.0.1:7860
```

The first launch downloads the ONNX files from Hugging Face (about 147 MB). After that the app works offline. Typical workflow:

1. In **Teach**, type a class name, upload 5-20 photos and press **Add examples**.
2. Repeat for at least one more class.
3. Press **Train model**; the status line reports image count and training accuracy.
4. Open **Classify** and upload, paste or take a webcam photo.
5. Use **Zero-shot** at any time to classify against typed labels.

11. Conclusion

Teachable Lens shows that a useful, personal image classifier does not need a GPU framework or a large labelled dataset. A quantized CLIP encoder running in onnxruntime supplies general-purpose features, and a logistic-regression head trained in milliseconds turns them into a classifier for any user-defined classes. On the synthetic benchmark it reached 99% accuracy with five examples per class and 100% with twenty, all within a laptop's CPU and memory budget.

References

[1] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision", ICML 2021.
[2] openai/clip-vit-base-patch32, Hugging Face Hub.
[3] Xenova/clip-vit-base-patch32, ONNX export, Hugging Face Hub.
[4] ONNX Runtime, https://onnxruntime.ai
[5] Gradio, https://gradio.app
[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", JMLR 12, 2011.